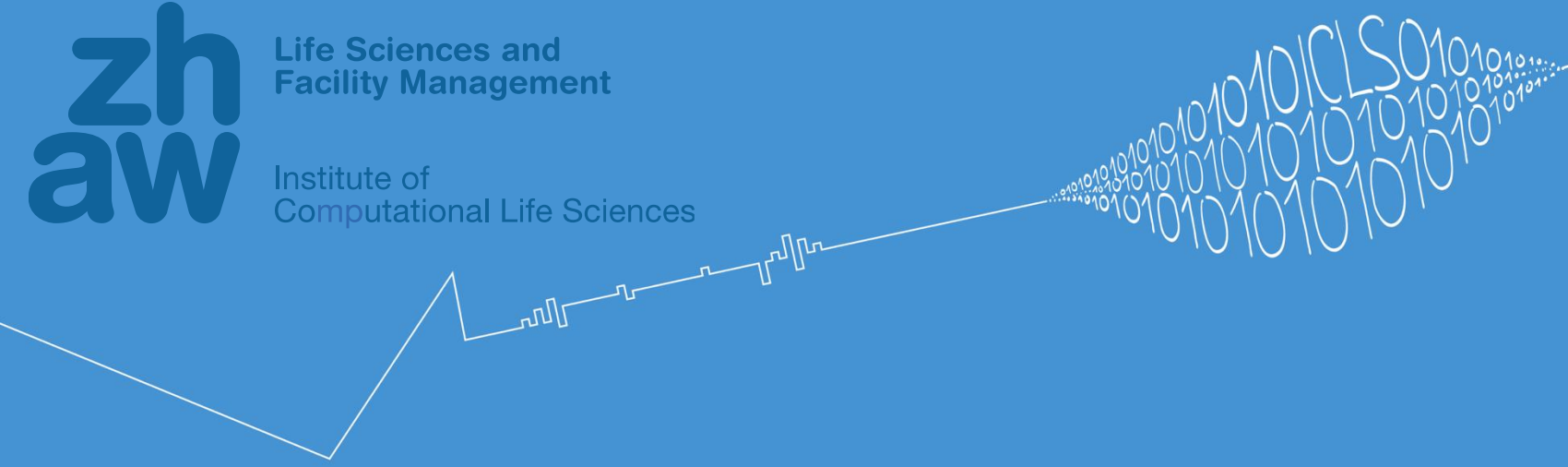
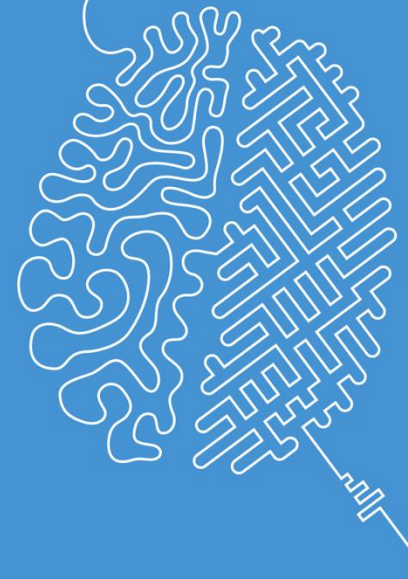
Life Sciences and
Facility Management

Institute of
Computational Life Sciences





Discrete Fourier Transformation (DFT)

Image and Signal Processing

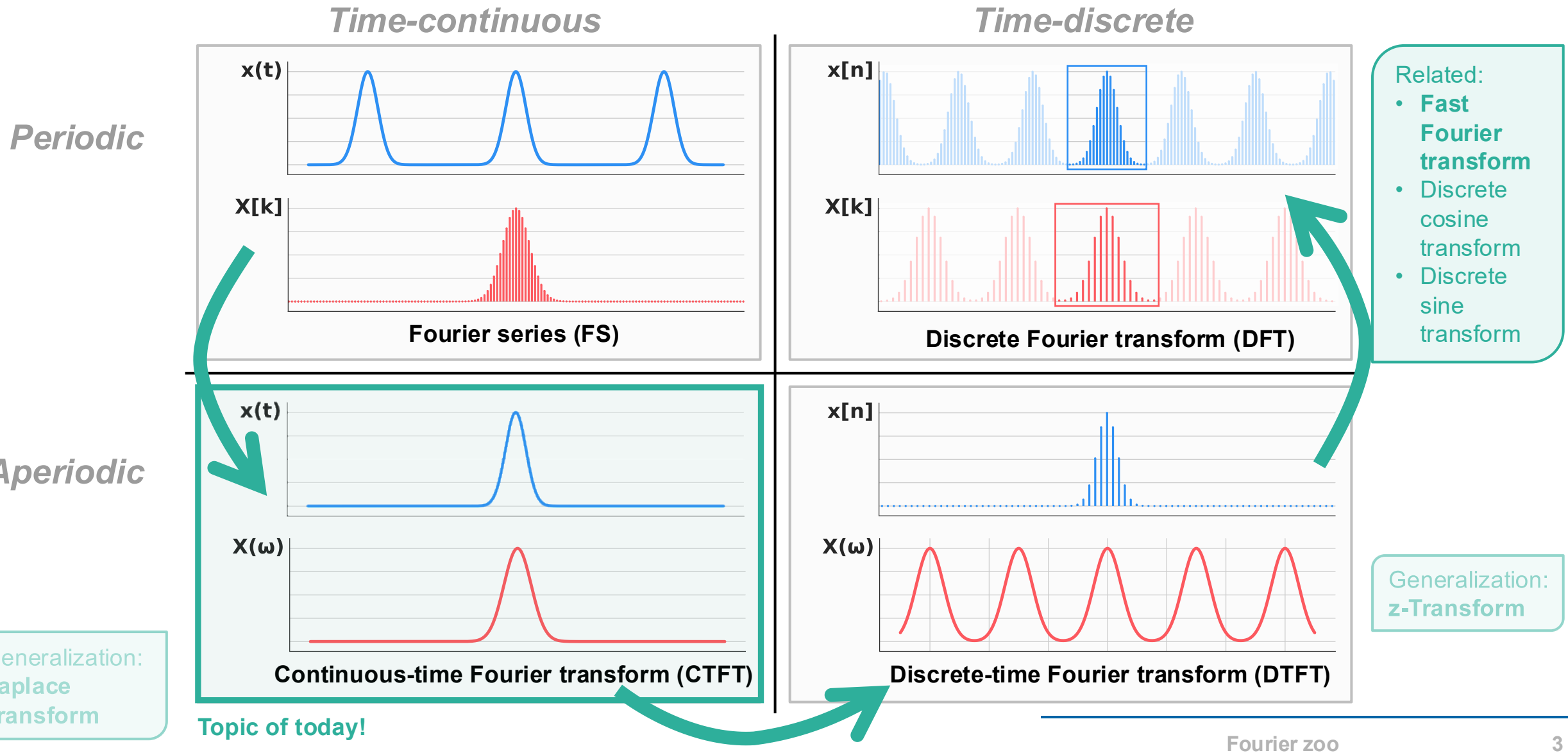
Norman Juchler

Fourier – Final episode

We are really wrapping this up now!

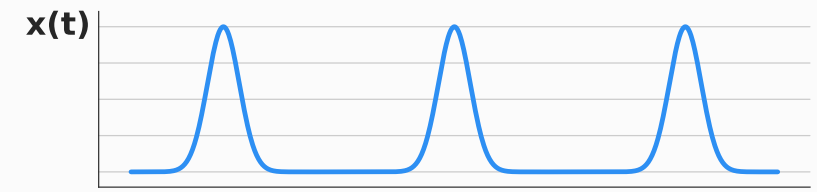
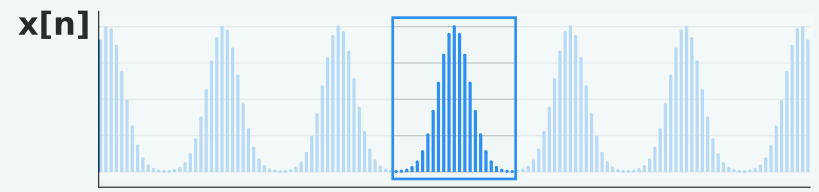
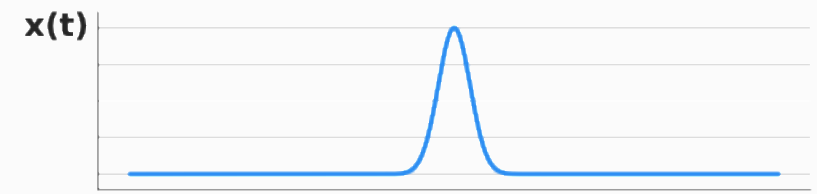
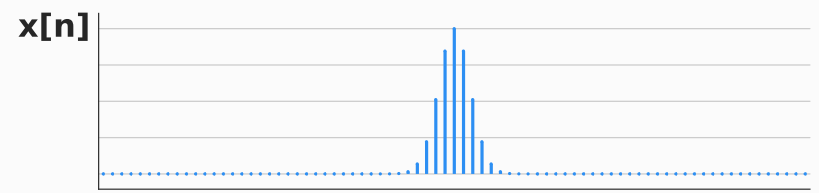
Fourier's landscape

Fourier series and the Fourier transform were originally developed for continuous-time signals. Here, we consider the discrete-time, finite-length counterparts that are used in digital signal processing.



Fourier's landscape

Key messages: A) Most concepts behave very similarly — except that what were previously integrals now become summations. B) In practice, the trafo we will focus on is the DFT, and its most widely used implementation, the FFT.

	Time-continuous	*Time-discrete*
Periodic	 **Fourier series (FS)**	 **Discrete Fourier transform (DFT)**
Aperiodic	 **Continuous-time Fourier transform (CTFT)**	 **Discrete-time Fourier transform (DTFT)**

Topic of today!

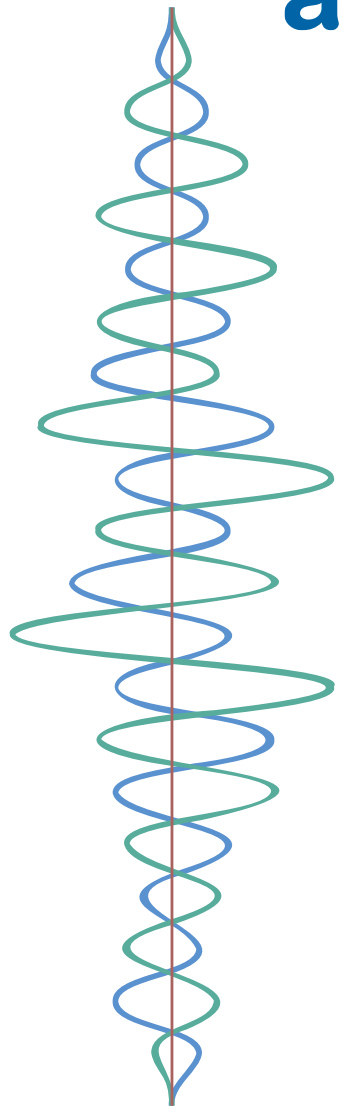
Related:
- **Fast Fourier transform**
- Discrete cosine transform
- Discrete sine transform

Generalization: **Laplace Transform**

Generalization: **z-Transform**

A zoo of Fourier transforms

- …but they share a common structure:
  - Each transform has a forward and an inverse transformation.
  - Many important mathematical properties are shared, including:
    - Linearity: $a \cdot x_1(t) + b \cdot x_2(t) \longleftrightarrow a \cdot X_1(\omega) + b \cdot X_2(\omega)$
    - Time (and frequency) shifting: $x(t - t_0) \longleftrightarrow X(\omega)\, e^{-j\omega t_0}$
    - The convolution theorem: $x(t) * h(t) \longleftrightarrow X(\omega) \cdot H(\omega)$
    - Scaling properties: $x(at) \longleftrightarrow \frac{1}{|a|} X\left(\frac{\omega}{a}\right)$
    - Energy preservation: $\int_{-\infty}^{\infty} |x(t)|^2\, dt = \frac{1}{2\pi} \int_{-\infty}^{\infty} |X(\omega)|^2\, d\omega$
- Take-home messages:
  - Fourier theory allows us to decompose signals into their fundamental frequency components.
  - It provides two equivalent representations of a signal: one in the time domain and one in the frequency domain.
  - Different variants of the Fourier transform are suited to different types of signals.



A zoo of Fourier transforms

Recall that from from Euler's formula it follows that:

$$e^{-i\varphi} = \cos(\varphi) - i\sin(\varphi)$$

Method	Input	Output	Forward Transform	Inverse Transform
FS	Continuous-time periodic signal $x(t)$ with period T	Discrete set of frequency coefficients c_k	$c_k = \frac{1}{T}\int_T x(t)\, e^{-ik\omega_0 t}\, dt,$ with $\omega_0 = \frac{2\pi}{T}$	$x(t) = \sum_{k=-\infty}^{\infty} c_k\, e^{ik\omega_0 t}$
CTFT	Continuous-time aperiodic signal $x(t)$	Continuous spectrum $X(\omega)$	$X(\omega) = \int_{-\infty}^{\infty} x(t)\, e^{-i\omega t}\, dt$	$x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(\omega)\, e^{i\omega t}\, d\omega$
DTFT	Discrete-time sequence $x[n]$	Continuous (2π-periodic) spectrum $X(\omega)$	$X(\omega) = \sum_{n=-\infty}^{\infty} x[n]\, e^{-i\omega n}$	$x[n] = \frac{1}{2\pi}\int_{-\pi}^{\pi} X(\omega)\, e^{i\omega n}\, d\omega$
DFT	Finite discrete sequence $x[n]$, $n = 0, \ldots, N-1$	Discrete spectrum $X[k]$	$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-i2\pi kn/N}$	$x[n] = \frac{1}{N}\sum_{k=0}^{N-1} X[k]\, e^{i2\pi kn/N}$

This table is provided mainly as an overview. Don't worry too much about the formulas. Instead, focus on recognizing the different variants and understanding how they differ in terms of their input–output behavior.

Discrete Fourier Transform

…and FFT

The discrete Fourier transform (DFT)

- **Input**: Finite discrete sequence $x[n]$ of N equally spaced samples of a signal
- **Output**: Finite discrete sequence of N equally spaced samples of the DFT

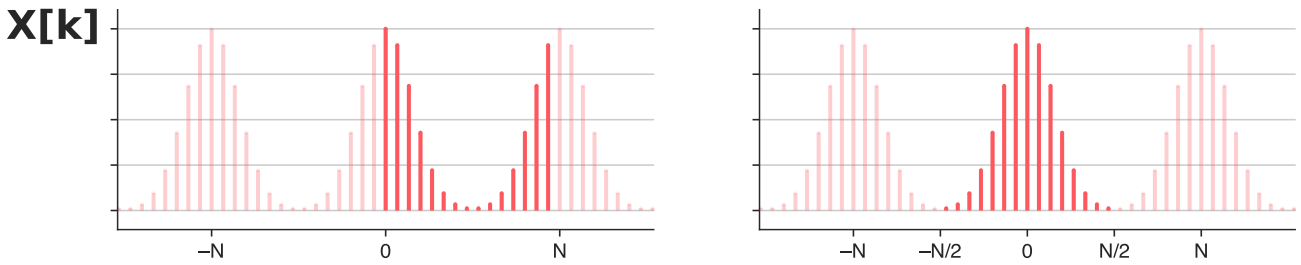
$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i\omega_k n}$$

Each coefficient X[k] represents the contribution of the frequency $f_k = \frac{k}{N} f_s$

$$\omega_k = 2\pi f_k = 2\pi \frac{k}{N} f_s, \quad k = 0, 1, 2, \ldots, N-1$$

$$\omega_k = 2\pi f_k = \begin{cases} 2\pi \frac{k}{N} f_s, & \text{if } 0 \le k < \frac{N}{2} \\ 2\pi \frac{k-N}{N} f_s, & \text{if } \frac{N}{2} \le k < N \end{cases}$$

Periodic extension



Note that, due to the periodicity of the DFT, the indices with $N/2 \le k < N$ correspond to the same frequencies as the indices in the range $-N/2 \le k < 0$.

The discrete Fourier transform (DFT)

- **Input**: Finite discrete sequence $x[n]$ of N equally spaced samples of a signal
- **Output**: Finite discrete sequence of N equally spaced samples of the DFT

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i\omega_k n}$$

Each coefficient X[k] represents the contribution of the frequency $f_k = \frac{k}{N} f_s$

$$\omega_k = 2\pi f_k = 2\pi \frac{k}{N} f_s, \quad k = 0, 1, 2, \ldots, N-1$$

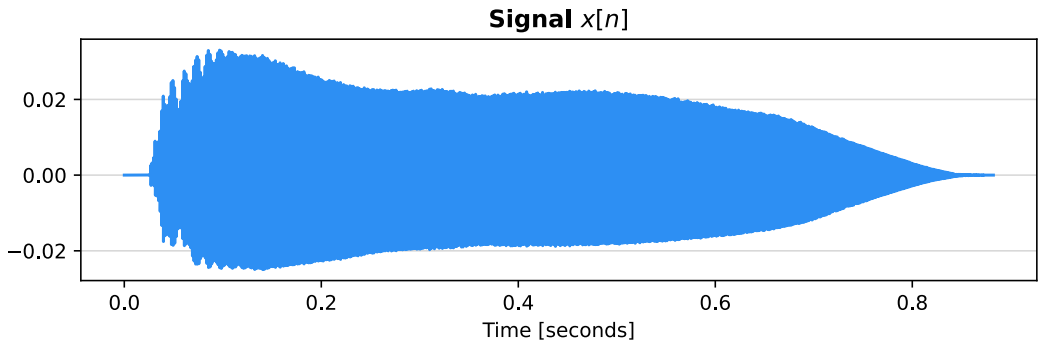
$$\omega_k = 2\pi f_k = \begin{cases} 2\pi \frac{k}{N} f_s, & \text{if } 0 \leq k < \frac{N}{2} \\ 2\pi \frac{k-N}{N} f_s, & \text{if } \frac{N}{2} \leq k < N \end{cases}$$

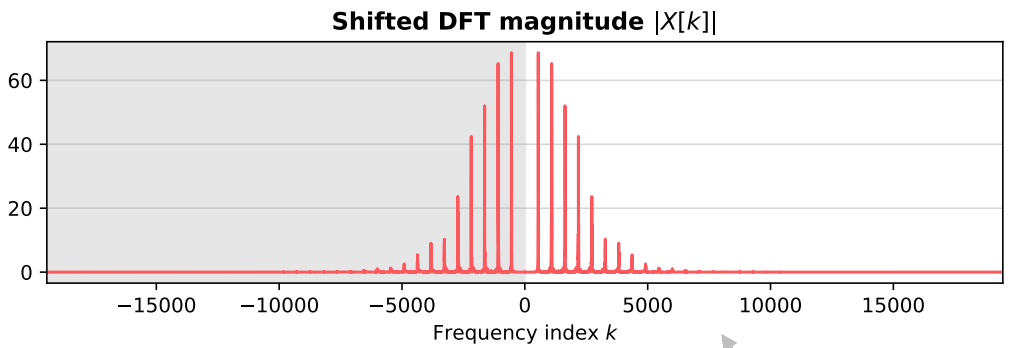
Periodic extension

- **Observations**:
  - In general, $X[k]$ are complex numbers that can be described by their magnitude and phase.
  - The DFT $X[k]$ is periodic with period N (a property leveraged, for example, by `fftshift()`).
  - Each coefficient $X[k]$ is associated with the frequency f_k.
    - Frequencies f_k with $k \geq N/2$ are typically interpreted as negative frequencies (periodic extension)
    - The first value $X[0]$ is called the **direct current** (DC) or static component, corresponding to $f_0 = 0$
    - $X[0]$ equals the sum of all sample values in the signal.

Example: A real sound

Sound of a trumpet recorded with a standard audio recorder. Source: Freesound





After applying `fftshift()`

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i\omega_k n} \qquad \omega_k = 2\pi f_k$$

$$f_k = \begin{cases} \frac{k}{N} \cdot f_s & \text{if } 0 \le k < \frac{N}{2} \\ \frac{k-N}{N} \cdot f_s & \text{if } \frac{N}{2} \le k < N \end{cases}$$

Look at the plots and answer the following **questions**:

- What is the sampling frequency?
- How many samples are there?
- What is the signal's DC value? How is it related to the mean value?
- At which frequency is the peak?

Answers:

- Sampling frequency: 44'100Hz
- Number of samples: 0.88[s]·44'100[1/s] ≈ 38'000
- Static component: first value X[0] = sum of x[n] ≈ 0
- Peak frequency: 618.8 Hz, corresponding to the played note $D^{\sharp 5}$

Example adapted from Brian McFee / Digital Signals Theory.

The discrete Fourier transform – a naive implementation:

```
1  def dft(x):
2      '''Compute the Discrete Fourier Transform of an input signal x of N samples.'''
3
4      N = len(x)
5      X = np.zeros(N, dtype=complex)
6
7      # Compute each X[k]
8      for k in range(N):
9
10         # Compute similarity between x and the k'th basis
11         for n in range(N):
12             X[k] = X[k] + x[n] * np.exp(-2j * np.pi * k * n / N)
13
14     return X
```

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-2i\pi kn/N}$$

Considering the effort required to derive the Fourier transform mathematically, the actual implementation of the DFT is surprisingly straightforward:

The **outer loop** iterates over all N frequencies, indexed by k. For each frequency, the **inner loop** computes the corresponding DFT coefficient by iterating over the N samples x[n] of the input signal.

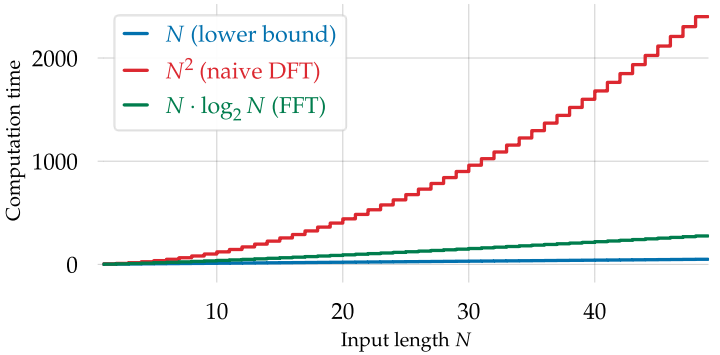
Question: How long does it take to compute the DFT for a signal x?

Answer: Line 12 needs to be calculated N^2 times, or using the big-O notation: The complexity of this DFT is $O(N^2)$

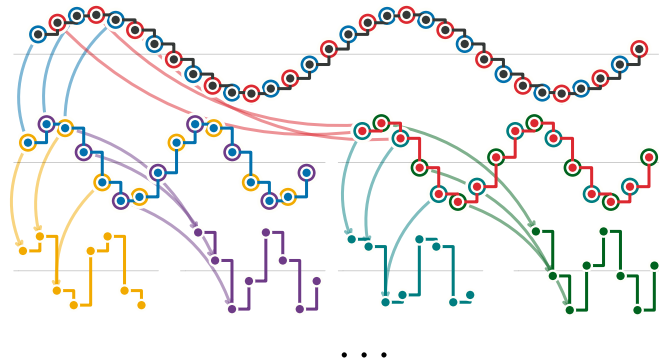
Code sample taken from Brian McFee / Digital Signals Theory

The Fast Fourier Transform (FFT)

- The FFT is an efficient algorithm to compute the DFT of a discrete signal
- History:
  - Some ideas of FFT can be traced back to C.F. Gauss (1805).
  - The algorithm was first published in 1965 by J. Cooley and J. Tukey.
  - The most popular version is called radix-2 Cooley-Tukey algorithm.
- Key idea: **Divide and conquer**
  - Exploit DFT symmetry to recursively break the computation into smaller sub-problems, eliminating redundant calculations.
  - The roots of unity play a central role here...
  - Instead of directly computing the DFT for $N = N_1 N_2$, it reformulates the problem in terms of for N_1 smaller problems of size N_2.
  - This approach reduces computational complexity from O(N^2) for a naive implementation to O(N log N)!



The increase of the amount of work (that is, the number of computations) increases with the problem size



A discrete signal of N=32 samples (top row) is divided into its even and odd samples (middle-left and middle-right).

Illustrations: Brian McFee / Digital Signals Theory

The Fast Fourier Transform (FFT)

```python
def fft(x):
    '''Compute the DFT of an input x of N = 2**m samples'''

    # Get the length of the input
    N = len(x)

    # The DFT of a single sample is just the sample value itself
    # Nothing else to do here, so return
    if N == 1:
        return x

    else:
        # Recursively compute the even and odd DFTs
        X_even = fft(x[0::2])  # Start at 0 with steps of 2
        X_odd = fft(x[1::2])   # Start at 1 with steps of 2

        # Allocate the output array
        X = np.zeros(N, dtype=complex)

        # Combine the even and odd parts
        for k in range(N):
            # Find the alias of frequency k in the smaller DFTs
            k_alias = k % (N//2)
            X[k] = X_even[k_alias] + np.exp(-2j * np.pi * k / N) * X_odd[k_alias]

        return X
```

Hearing about the FFT might have suggested that the algorithm would be quite complicated. However, its implementation again turns out to be surprisingly straightforward.

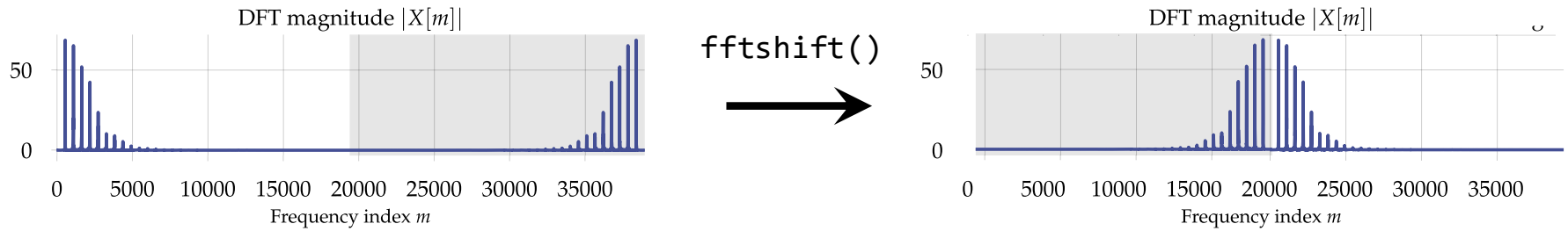
Note the **recursive structure** of the algorithm. The function fft() calls itself to compute the transforms of the even-indexed and odd-indexed samples of the signal. This is a classic example of a **divide-and-conquer** strategy: a large problem is split into smaller subproblems, which are solved recursively.

The results of the even and odd parts are then combined, implicitly making use of the **symmetry** and **periodicity** properties of the DFT.

Note that we could easily improve the performance of this simple implementation by vectorizing the loop, as we would typically do when working with NumPy arrays. That said, in practice we usually rely on highly optimized FFT libraries, which exploit advanced hardware features.

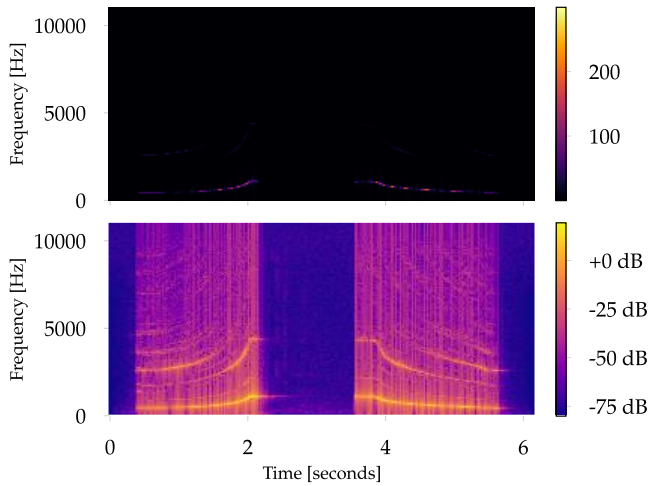
The Fast Fourier Transform (FFT)

- The scipy.fft package provides fast implementations of FFT algorithms
- See this tutorial for for usage examples.
- Core functions:
  - `fft(), fft2(), fftn():` Compute the FFT for 1D, 2D, or nD input.
  - `ifft(), ifft2(), ifftn():` Compute the inverse DFT in 1D, 2D or nD.
  - `rfft(), rfft2(), rfftn():` Compute the FFT – optimized for real-valued input.
  - `irfft(), irfft2(), irfftn():` Inverse transform for real-valued spectra.
  - `fftshift():` Rearrange FFT output so that the zero frequency is centered.
  - `fftfreq(), rfftfreq():` Return the frequency bins corresponding to FFT output.

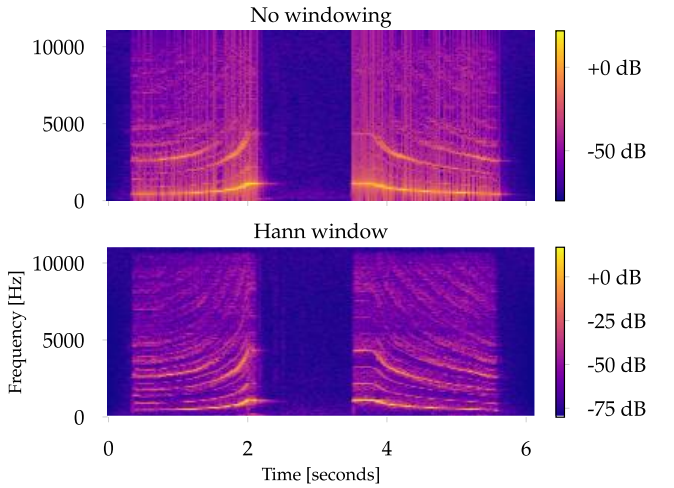


The short-time Fourier Transform (STFT)

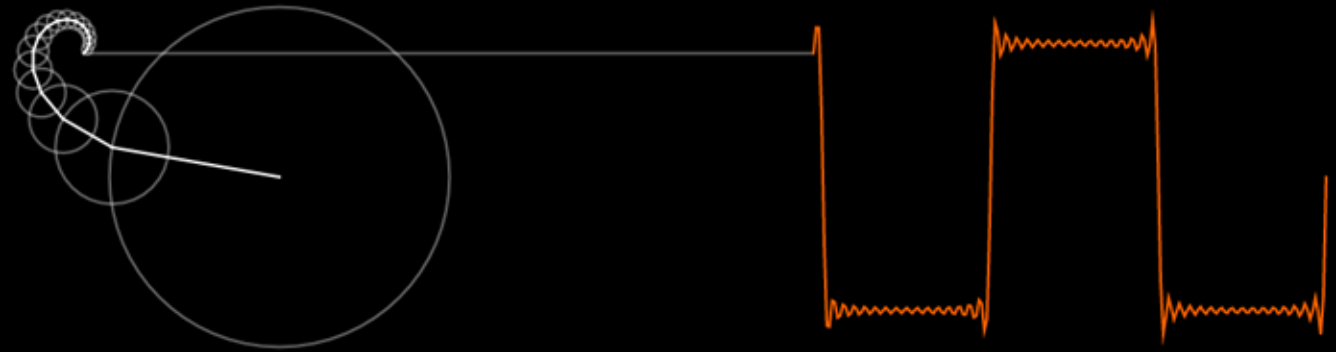
- **Problem**:
  - The DFT uses all samples of a signal at once
  - This implicitly assumes the frequency content is stationary
  - For time-varying signals, this assumption often fails
- **Solution**: Divide the signal into short segments (**frames**) and analyze each frame separately.
- Key parameters:
  - Frame length (number of samples per frame)
  - Hop length (number of samples between two frames)
- STFTs can be visualized with **spectrograms**
  - Created by stacking the spectra of successive frames
- Practical hints:
  - Use a decibel scale for better visualization
  - Apply windowing (“apodization”) to reduce artifacts



Spectrogram with linear (top) and decibel (bottom scaling ($A_{\mathrm{dB}} = 20 \cdot \log_{10}(A)$). Notice the improved visibility of spectral patterns.



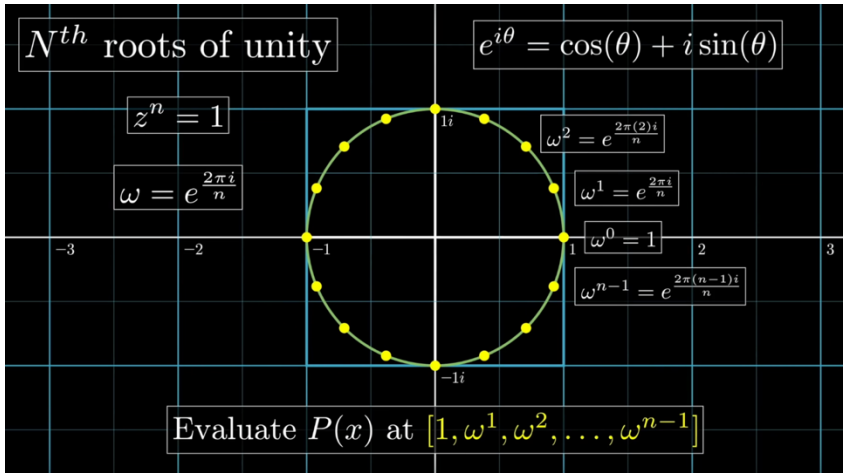
Spectrogram without windowing (top) and with windowing (bottom). A window function (here: Hann) attenuates discontinuities at the frame boundaries.



Source: Greg Lund / FFT drawer

Recommended videos

- Reducible / 2023: The Fast Fourier Transform (FFT): Most Ingenious Algorithm Ever? Link



- Veritasium / 2023: The Most Important Algorithm Of All Time. Link